

80/20 Experiment Metadata Schema

Philosophy

This document describes the metadata schema for quantum algorithm experiments in the 80/20 benchmarking study. The schema captures “who, what, where, when”, a little bit of the elusive “why”, and “how” for each experiment run. Runs are a case, embodied in a code i.e. algorithm, and executed on what is typically called a backend.

Here “80/20” refers to a rule of thumb regarding quantum utility - when you can get results at least 80% as good as classical on a problem at least 20% of a useful size, then utility is in view. This isn’t to be taken literally, and we’re not basing the quantities on anything other than human instinct. Note that speed to solution is ignored in this formula - “build it, then make it fast” is a common mantra in software.

We additionally accept up front that due to the variability in cases, codes, and backends, and the high rate of change in quantum research, that no one metadata schema is going to work for all of them. Traditionally, some kind of “ETL” process might be used to laboriously ingest and normalize heterogeneous data. While not altering that approach, we gain productivity by leaning on the modern language-based AI software engineering tooling. This allows us to adopt a flexible metadata scheme.

While originally intended for a CFD study, there’s nothing specific to CFD here, or any domain or algorithm / code in particular, and one does not have to agree with the “80/20” rule of thumb. Quantum hardware is changing fast, becoming obsolete quickly, and research-grade software stacks get abandoned. The ability to reproduce a result even a few months in the future is unlikely. Hardware calibration changes often - hourly even - and these can also be important for interpreting the results now and in future.

Exact reproducibility of the experiment is therefore out of reach and is not a goal. Approximate reproduction is a second tier goal. The top objective here is comparability over time - years potentially - where successive results which individually do not demonstrate utility together illustrate the trend in a particular direction - some case(s), solved by certain code(s), on specific backend(s).

The intent is simply to capture as much metadata as possible about a quantum computing experiment, the scientific setup, the environment in which it ran (e.g. compiler / language version, libs), and on which it ran. Capture then permits comparison, which implies search. All the principles of F.A.I.R. data management apply here.

Capturing Metadata: Open & Closed Boxes

There will be variability in what can be captured for many reasons. Open box codes can be heavily instrumented - we provide a Python library to make this convenient. For example, a method which takes an IBM backend and mines it for all its calibration and connection data, or one which does similar with a transpiled circuit, or one which takes a snapshot of all the versions of the libraries in scope for the run.

Closed box codes are harder to gather from, but any discernable output is potentially queryable by a metadata “harvester” armed with even a minimal natural language knowledge of the expected output. The modern LLM tooling is a real benefit in this regard - in gathering data from parsed output files, in validating contents are complete as intended, in morphing between equivalent formats.

Closed boxes also lack a certain immediacy if metadata gathering is not in time with the run - if metadata is harvested later, installed library versions might change, backends might too. We do the best as we can, as close to the moment as possible.

For quantum backends, the discernable data about the machine may differ by vendor, or at least names may differ at minimal trivially. For coded algorithms, there will be obvious differences in input arguments and many other factors. Cases, even in a specific domain (e.g. CFD) will also differ widely in their descriptions. There will be no attempt to unnaturally shoehorn all the above into a single fixed schema. To borrow from musical production parlance, we will record the raw tracks and “fix it in the mix”.

Tooling

This repo contains some reusable tools and all should at the moment be considered “works in progress”, provided as-is.

- `args.py`: boilerplate arguments for common quantum codes (e.g. backend, shots)
- `meta_fragment.py`: takes objects and produces metadata json, fragments being different aspects of the metadata set (e.g. metadata fragment about the run environment / backend)
- `harvest.py`: rolls up fragments for an experiment (i.e. specific {case, code, backend}). Note that some fragments could potentially appear multiple times in the same experiment.
- `compare.py`: what it implies, work-in-progress
- `metakeys.py`: find the metadata in an experiment or common across experiments, flatten the name tree or be verbose; necessary for search and comparison, work-in-progress
- `sweep.py`: take a TOML describing the experiment and run it, “sweeping” parameters where specified in the TOML. Permits pre- and post-processing, use of venvs, subdir per experiment, etc. Current work-in-progress is to extend this to leverage HPC (i.e. slurm interface) for (embarrassing at least) parallelism.

Repos

Besides this one, the “q8020-cfd-axequalsb” repo contains some open box codes used in the CFD study - this is a growing set, and does not include closed box codes stored elsewhere.

“q8020-cfd-experiments” is where we are storing the harvested experiment metadata for the CFD project. “q8020-cfd-docs” describes the project. “q8020-cfd-qutil” is a small collection of convenience utilities (e.g. to remove “q8020-cfd-metadata” from having dependency on Qiskit, functions to poll IBM backends are here).

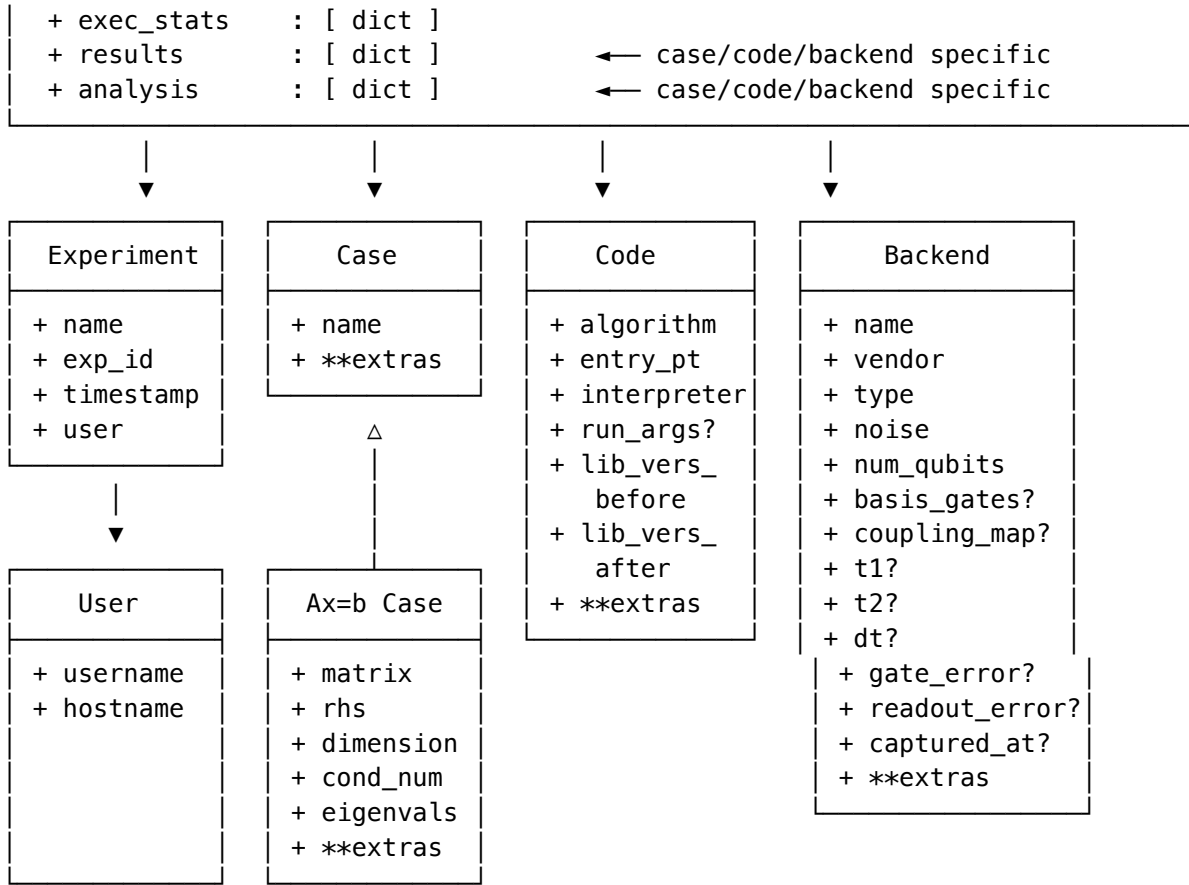
MACHINE-GENERATED DOC

Top-Level Metadata Categories

The goal is comparability across like experiments. In human terms:

- Who: User (username, hostname)
- What: Case (problem definition), Results (solutions)
- Where: Backend (execution environment), Code (library versions)
- When: Experiment (timestamps), Exec Stats (timing)
- Why: Elusive—human-provided context outside this schema - any user-provided fields are permitted to give additional context
- How: Code (entry point, algorithm), Artifacts (circuit details)

ExperimentOutput		
All sections are JSON arrays (lists). Typically one element each for experiment/case/code/backend, but the schema permits multiple fragments.		
+ experiment	: [Experiment]	
+ case	: [Case]	← case/code/backend specific
+ code	: [Code]	← case/code/backend specific
+ backend	: [Backend]	← case/code/backend specific
+ artifacts	: [dict]	← case/code/backend specific



All sections are harvested as arrays — even those that typically have a single element (experiment, case, code, backend). The ? suffix indicates optional fields. **extras indicates extensible fields that vary by case/code/backend.

Top-Level Fields

Field	Required	Description	Variations
experiment	Yes	Run identification: name, IDs, timestamp, user	Stable
case	Yes	Problem definition	Case-specific (e.g., matrix/rhs for Ax=b)
code	Yes	Algorithm and environment	Code-specific (library versions vary)
backend	Yes	Execution target	Backend-specific (noise model, topology)
artifacts	No	Circuit info, transpilation, files	Code/backend-specific
exec_stats	No	Timing, exit status	Stable
results	No	Solutions (classical, quantum)	Case/code-specific
analysis	No	Computed metrics (fidelity, error)	Case/code-specific

Where Variations Occur

Case-Specific

The case section defines the problem being solved. Structure depends entirely on the problem type: - Linear systems ($Ax=b$): matrix, rhs, dimension, condition_number, eigenvalues - Optimization: objective function, constraints, initial parameters - Simulation: Hamiltonian, time evolution parameters

Code-Specific

The code and artifacts sections vary by algorithm: - HHL/HHL-Qrisp: QPE precision, Trotter steps, Hamiltonian simulation details - VQLS: Ansatz configuration, optimizer settings, iteration counts - CKS: Block encoding parameters

Backend-Specific

The backend section varies by vendor, by execution target: - Ideal simulator: Minimal—just name and qubit count - Noisy simulator: Noise model, coupling map, gate errors, readout errors - Real hardware: All of the above plus calibration data, T1/T2 times

Field Details

experiment

Identification and provenance. Stable across all experiment types.

Field	Type	Description
name	string	Experiment/case name from config
experiment_id	string	Unique 8-hex identifier
workflow_id	string	Parent sweep/workflow ID
timestamp	string	ISO 8601 UTC
user	object	{username, hostname}

case

Problem definition. Base fields plus case-specific extensions.

Field	Type	Description
name	string	Problem type identifier
varies	varies	Case-specific fields

code

Algorithm and execution environment.

Field	Type	Description
algorithm	string	Algorithm identifier (hhl, vqls, cks, etc.)
entry_point	string	Script path
interpreter	string	Runtime (python, etc.)
run_args	dict?	Algorithm-specific configuration (key→value)

Field	Type	Description
library_versions_before	object?	Package→version snapshot captured before execution
library_versions_after	object?	Package→version snapshot captured after execution

backend

Execution target details.

Field	Type	Description
name	string	Backend identifier
vendor	string	Provider (ibm, etc.)
type	string	simulator or hardware
noise	boolean	Whether noise model applied
num_qubits	int	Available qubits
basis_gates	array?	Native gate set
coupling_map	array?	Qubit connectivity
t1	object?	Per-qubit T1 relaxation times (μs)
t2	object?	Per-qubit T2 dephasing times (μs)
dt	float?	Backend sample time (seconds)
gate_error	object?	Per-gate error rates
readout_error	object?	Per-qubit readout errors
captured_at	string?	ISO 8601 timestamp of when backend data was captured

artifacts

Circuit and execution details. Highly code/backend-specific.

Common fields: - transpile_passes: Before/after gate counts, depth, optimization level - circuit_construction_time_s: Build time - execution_time_s: Run time - exec_info: Shots, backend time, status

exec_stats

Execution statistics. Stable across all types.

Field	Type	Description
start_time	string	ISO 8601 start
end_time	string	ISO 8601 end
duration_seconds	float	Total time
exit_code	int	Process exit code
success	boolean	Completion status

results

Solution data. Case/code-specific.

For linear solvers: | Field | Type | Description | |———|———|———| | classical_solution | array | Exact solution $x = A^{-1}b$ | | classical_solution_normalized | array | Unit-normalized | | quantum_solution | array | Quantum result | | quantum_solution_normalized | array | Unit-normalized |

analysis

Computed comparison metrics. Case/code-specific.

Common fields: | Field | Type | Description | |-----|-----|-----| | fidelity | float | State fidelity | | l2_error | float | L2 norm error | | residual | float | $\|Ax - b\|$ |

Workflow-Level Rollup

Per-case metadata rolls up into a workflow-level summary containing: - Config file path - Workflow ID and output directory - Start/end timestamps - Full library versions snapshot - Per-case status, timing overhead, commands

Data Sources

Each field carries a `_source` tag indicating provenance: - "solver": Emitted by the algorithm script - "sweep": Added by the experiment harness - "stdout": Parsed from script output

This enables tracing where each piece of data originated.

Sweeper-Level Metadata

The sweeper (experiment harness) maintains its own metadata separate from per-case experiment data. This captures the orchestration context for a batch of experiments.

Sweep Output File

The sweeper writes `q8020_sweep_meta_<workflow_id>.json` to the run directory with:

Field	Type	Description
config_file	string	Path to input TOML configuration
output_dir	string	Base output directory
workflow_id	string	Unique 9-char identifier for this sweep
run_dir	string	Full path to this run's output
timestamp	string	Run timestamp (YYYYMMDD_HHMMSS)
start_time	string	ISO 8601 sweep start
end_time	string	ISO 8601 sweep end
sweeper_library_versions	object	Package versions captured by sweeper
groups	object	Per-group postproc results
cases	object	Per-case execution results (keyed by experiment_id)

Per-Case Execution Record

Each entry in `cases` contains sweeper-observed execution data:

Field	Type	Description
command	array	Full command executed
status	string	success, error, timeout, exception
returncode	int	Process exit code
start_time	string	ISO 8601 case start
end_time	string	ISO 8601 case end

Field	Type	Description
duration_seconds	float	Wall-clock time
overhead	object	Sweeper timing (preproc, postproc, harvest)

Relationship to Case Metadata

The sweeper metadata and per-case `q8020_metadata_<experiment_id>.json` are complementary:

- Sweeper metadata: External view—what commands ran, exit codes, timing overhead
- Case metadata: Internal view—algorithm parameters, results, analysis

The `experiment_id` links records across both files. The sweeper also copies the input TOML and writes `q8020_expanded_cases.json` showing the full parameter expansion before execution.